

Modularity Across Cross-Partner Compositions: Formalizing How CKS Modular Architecture Operates Across Deployment Partners per A1.16 and A2.47

Derivation Note B2.38 — Phase B2, Series B

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 12, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the instinct/reasoning separation pattern introduced in “The Instinct/Reasoning Separation Outside the Model” (Li, April 2026), the second paper in the CKS theory series following “Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems” (Li, April 2026).

Abstract

CKS modular architecture, established by B1.08 as an architectural commitment rather than an emergent property, extends to multi-partner deployment arrangements through standard modular interfaces that apply regardless of which deployment partner originated a given cell or aspect. This note formalizes that extension as a standalone operational variant. The A1.02 modular boundary is the same for all CKS cells regardless of partner origin; the A1.13 composition requirements apply to all cross-partner compositions without modification; the A1.16 hybrid-systems composition patterns (Pattern A consultation, Pattern B derived view, Pattern C separate concern) apply at the cross-partner scope through A2.92, A2.93, and A2.94. Authority distribution per A2.47 (Category 5) is the foundational governance mechanism for shared components: without established cross-partner authority, shared components cannot be governed. A6.12 multi-author rule authoring conflict treatment applies when cross-partner rule authoring produces conflicts. Each deployment partner retains their A1.01 governance over their own components; shared components require cross-partner authority per A2.47. The governance model is federated. Cross-partner composition through standard interfaces is architecturally distinct from proprietary multi-party integration, which requires per-pair negotiation and custom code at every integration point. This note is the fourth of five decompositions of B1.08 in Phase B2 (following B2.35, B2.36, B2.37; B2.39 closes the decomposition).

1. Why modularity across cross-partner compositions requires standalone formalization

B1.08 establishes that CKS modularity is an architectural commitment: cells are modular units by design, composition of cells into aspects and Selves follows specified composition requirements (A1.13), and modularity enables configurability, vertical evolution, and reuse without requiring a re-specification of the modular interface at each new use. Prior decomposition notes in this series have formalized modularity as an architectural consequence of the CKS commitments (B2.35), as the basis for composition configurability (B2.36), and as the enabler of vertical evolution without full-system replacement (B2.37).

Each of those treatments operates within a single deployment partner's scope. B2.38 addresses the extension that B1.08 modularity makes available but that none of B2.35 through B2.37 directly formalizes: the operation of CKS modular architecture across composition partners. When two or more deployment partners each operate CKS deployments and seek to compose cells or aspects from their respective deployments into shared structures, the modular architecture that B1.08 commits to must extend across the partner boundary without requiring either partner to negotiate a custom integration interface.

The reason this extension needs to be formalized as a standalone variant is that it introduces a governance dimension that single-partner modularity does not carry. Within a single partner's deployment, authority over cells, aspects, and orchestration rules is held by that partner's humans under A1.01. Across partners, shared components require shared authority — and that shared authority must be established explicitly, governed explicitly, and its scope specified explicitly, before cross-partner operations can proceed. The architectural mechanism that makes this governable is A2.47 (Category 5) authority distribution. Without naming this mechanism explicitly, cross-partner composition is architecturally underspecified, and the prior-art chain covering multi-partner CKS deployments has a gap.

As the thirty-eighth note in Phase B2 and the fourth in the B1.08 decomposition, B2.38 occupies a specific position in the defensive-publication strategy: it places into the public record, before any claim of novel invention can be filed, the precise architecture by which CKS modular interfaces support cross-partner composition under federated governance.

2. The architectural specification

Standard modular interfaces apply regardless of partner origin. The A1.02 modular boundary is defined at the CKS architecture level, not at the partner level. A cell from Partner A's deployment and a cell from Partner B's deployment both carry the same modular interface: the same boundary between substrate layer and cell layer, the same separation between what the substrate commits to and what the cell commits to, and the same channel structure through which cells interact with the substrate. Because the interface is architecturally fixed, cells from different partners can compose into shared aspects without either partner building a translation layer.

The same uniformity holds for aspects. The A1.13 composition requirements that govern how cells compose into aspects specify what any aspect composition must satisfy. Those requirements do not contain a partner-identity field; they apply to any composition of CKS cells, regardless of the cells' origin. An aspect that draws cells from both Partner A and Partner B satisfies the composition requirements if and only if it satisfies A1.13 — the same five requirements that govern a single-partner aspect composition.

Cross-partner composition patterns per A2.92, A2.93, A2.94. The A1.16 hybrid-systems composition patterns — Pattern A (adjacent component as input), Pattern B (adjacent component as derived view), Pattern C (adjacent component as separate concern) — apply at the cross-partner scope through the A2.92–A2.94 cross-partner rendering. Pattern A at cross-partner scope is consultation: Partner A's cells consult Partner B's cells during execution, with Partner B's substrate remaining non-authoritative on Partner A's coordination questions. Pattern B at cross-partner scope is derived view: a shared aspect derives a view from cells drawn from multiple partners, with the constituent substrates remaining authoritative and the shared aspect held to A1.13's composition requirements. Pattern C at cross-partner scope is separate concern: Partner A's aspects and Partner B's aspects coexist in a shared Self with distinct concerns and no architectural coupling between the partners' respective governance domains.

Each of these patterns has sharpening requirements per A4.27, A4.28, and A4.29. The sharpening properties specify what must be true for cross-partner composition under each pattern to satisfy the A1.13 composition requirements and preserve each partner's A1.01 governance. The patterns are not self-satisfying; they require explicit specification of which pattern applies to each cross-partner arrangement before that arrangement can be governed correctly.

Authority distribution per A2.47 (Category 5) is foundational. A2.47 specifies who has authority over which components in CKS deployments, including the category of shared components in cross-partner arrangements. The architectural commitment is explicit: cross-partner authority must be established before cross-partner operations proceed. A deployment that attempts cross-partner composition without establishing A2.47-compliant cross-partner authority for shared components is architecturally underspecified; shared components in such a deployment have no governed authority structure, which means neither partner can exercise A1.01 governance over those components in a determinate way.

Cross-partner membership — the registration of cells or aspects from one partner into a shared structure also populated by another partner's components — requires cross-partner authority. Cross-partner rule authoring — the authoring of orchestration rules that govern shared aspects — requires cross-partner authority. These requirements are not softened by the modular architecture; the modular architecture makes cross-partner composition structurally possible, but governance authority over shared components must be established separately and explicitly.

A6.12 multi-author rule authoring conflict treatment. When Partner A authors an orchestration rule for a shared aspect and Partner B authors a conflicting orchestration rule for the same aspect, A6.12 conflict treatment applies. The conflict is preserved as first-class substrate state per A1.03; neither partner's rule silently overrides the other's. The A6.12 boundary case is the cross-partner rendering of the conflict-as-first-class commitment: cross-partner rule authoring conflicts are architectural artifacts, not error states requiring immediate resolution by the substrate. Resolution is a governed operation under the cross-partner authority A2.47 establishes.

Cross-partner provenance per A2.40. All six provenance metadata fields per A2.40 apply to cross-partner operations. Cross-partner substrate writes carry partner attribution as part of their provenance record. Cross-partner rule authoring carries authorship attribution traceable to the partner whose humans authored the rule. This makes cross-partner audit possible: an auditor can trace any cross-partner operation to the partner who performed it, the authority under which it was performed, and the substrate state it produced.

3. What makes modularity across cross-partner compositions architecturally distinctive

The defining contrast is with conventional multi-party AI architectures. When two organizations each deploy AI systems with proprietary substrate designs and attempt to compose those systems into shared structures, they must negotiate a custom integration interface. The negotiation covers representation format, authority semantics, conflict handling, and provenance recording. The result is an integration layer that is specific to the two parties and to the integration's purpose; a different pair of organizations, or the same organizations pursuing a different integration purpose, must renegotiate and rebuild.

CKS modular architecture provides standard interfaces at the architecture level. The A1.02 boundary, the A1.13 composition requirements, the A1.14 adjacency distinctions, and the A1.16 patterns apply to every CKS deployment by architectural commitment. Two organizations that each operate CKS deployments share, by virtue of that architecture, a common modular interface for cells and aspects. The integration

question shifts from “how do we build a compatible interface” to “how do we establish cross-partner authority per A2.47 and which A2.92–A2.94 pattern applies to each shared structure.” The architectural work is configuration and governance specification, not custom interface engineering.

This reduction in integration friction is architecturally grounded, not incidental. It follows directly from B1.08’s commitment to modularity as architectural requirement rather than emergent property: because every CKS cell is modular by commitment, not by accumulated engineering effort, cross-partner cells are immediately interface-compatible. The standard interface is not a negotiated product; it is a consequence of the architecture both parties instantiate.

Authority distribution per A2.47 provides the explicit governance framework that makes this reduction in friction compatible with each partner’s A1.01 governance commitments. Without A2.47’s explicit authority-distribution mechanism, standard interfaces would allow cross-partner composition to proceed without any specified authority structure for shared components — which would mean neither partner could exercise meaningful governance over those components. A2.47’s Category 5 distribution supplies the governance layer that makes the standard interface architecturally complete rather than merely structurally convenient.

4. Inherited Paper 1 commitments

Cross-partner modularity inherits all six Paper 1 commitments through Paper 2’s modularity framework. A1.01 governance is exercised per partner over each partner’s own components; shared components are governed through the cross-partner authority A2.47 establishes. A1.02 modular boundary is the same for all CKS cells regardless of partner origin — this is the interface uniformity that makes cross-partner composition possible. A1.03 conflict-as-first-class applies to cross-partner conflicts, including A6.12 multi-author rule authoring conflicts; cross-partner conflicts are substrate-level artifacts with provenance, not error states. A1.13 composition requirements apply to cross-partner aspect and Self compositions without modification; the five requirements do not distinguish partner origin. A1.16 hybrid-systems patterns apply at cross-partner scope through A2.92–A2.94. A2.04 rule authoring applies to cross-partner rule authoring, with A6.12 treatment for conflicts between partner-authored rules. A2.40 six provenance metadata fields apply to cross-partner operations. A1.14 adjacency distinctions apply to cross-partner hybrid compositions, distinguishing CKS cross-partner composition from non-CKS multi-party AI integration at the architecture level.

5. The federated governance model

The governance model for cross-partner compositions is federated. Each partner governs their own components under their own A1.01 authority: each partner’s humans retain inspect, modify, and override rights over that partner’s cells, aspects, and orchestration rules. This per-partner governance is not modified by cross-partner composition; a cell that participates in a shared cross-partner aspect does not thereby exit its originating partner’s governance scope.

Shared components — aspects that draw cells from multiple partners, orchestration rules that govern those shared aspects, Selves that incorporate aspects from multiple partners — require cross-partner authority per A2.47. Cross-partner authority specifies which humans, from which partners, hold which authority over shared components. The authority distribution is explicit substrate content: it is inspectable, modifiable, and overridable by the humans who hold cross-partner authority, under the same A1.01 properties that govern any CKS substrate content.

The federated model distinguishes two authority domains: per-partner authority over partner-origin components, and cross-partner authority over shared components. Neither domain automatically extends into the other. Partner A's authority over Partner A's cells does not automatically extend to shared aspects; cross-partner authority over shared aspects does not automatically extend to the partner-origin cells that participate in those aspects. The boundary between the two authority domains is itself substrate content, specified under A2.47 and auditable through A2.40 provenance.

6. Operational implications

Configure A2.47 cross-partner authority as the foundational step. A deployment that intends cross-partner composition must establish cross-partner authority before initiating cross-partner operations. The authority specification names which components are shared, which partners hold authority over which shared components, and under what conditions cross-partner operations may proceed. This is a governance design step that precedes architectural composition, not a configuration detail that can be deferred until after shared structures are built.

Cross-partner membership and rule authoring require cross-partner authority. Adding a cell from Partner B to a shared aspect requires cross-partner authority over that aspect. Authoring an orchestration rule that governs a shared aspect requires cross-partner authority over that aspect's orchestration rules. Vertical evolution per B1.16 at cross-partner scope — reorganizing cells across aspect boundaries, retiring shared aspects, creating new shared aspects — requires cross-partner authority for the reorganization. None of these operations proceeds without the authority A2.47 establishes.

A6.12 conflict treatment governs cross-partner rule authoring conflicts. When both partners author rules for the same shared aspect, conflicts are treated per A6.12: preserved as first-class substrate state, with provenance attributing each conflicting rule to its authoring partner. Resolution is a governed operation, not automatic. This property keeps the cross-partner conflict record clean: neither partner can inadvertently extinguish the other partner's rule authoring by writing a conflicting rule.

Cross-partner provenance enables cross-partner audit. Because A2.40 provenance metadata applies to cross-partner operations, an auditor can reconstruct the full history of a shared component: which partner contributed which cells, which partner authored which orchestration rules, under whose cross-partner authority each operation proceeded, and when cross-partner authority was granted or modified. The audit capability is a direct consequence of the provenance commitment operating at cross-partner scope.

Many cross-partner arrangements are possible across different deployment contexts. The architecture does not prescribe specific cross-partner relationships. A deployment may participate in multiple cross-partner arrangements with different partners, each under different A2.47 authority specifications, for different shared structures, in different deployment contexts. The standard modular interface permits this flexibility; the A2.47 authority mechanism governs each arrangement specifically.

7. Limits

Standard interfaces do not eliminate the need to establish A2.47 authority. Interface uniformity makes cross-partner composition structurally possible but does not make it automatically governed. Each cross-partner arrangement requires an explicit A2.47 authority specification before it is architecturally complete.

Cross-partner composition does not confer unlimited cross-partner authority. A2.47 authority distribution is specific to named shared components. A partner that holds cross-partner authority over a shared aspect does not thereby acquire authority over the other partner's non-shared cells, orchestration rules, or governance structure. Authority distribution is bounded; cross-partner authority grants governance over specified shared components only.

Cross-partner modularity does not imply automatic trust. Governance is explicit throughout. The fact that Partner A's cells and Partner B's cells share the same modular interface does not create any implied authority relationship; A2.47 authority must be established deliberately, and its scope is exactly what the authority specification states.

Cross-partner modularity does not prescribe specific partner relationships. The architecture specifies the governance and interface mechanisms; it does not specify which organizations should be composition partners, what the scope of any particular cross-partner arrangement should be, or how cross-partner authority should be allocated between partners in any specific case. These are deployment decisions made under the architecture's governance framework, not architectural decisions made by the architecture.

Cross-partner composition may require agreements beyond architectural specification. The architectural commitments formalized here describe what must be true at the architectural level for cross-partner composition to satisfy A1.01 and A1.13. Whether the partners have legal, contractual, or operational agreements that support or constrain those architectural commitments is outside the scope of this note. Architectural specification and legal agreement are distinct layers; both may be necessary in practice.

Cross-partner modularity is not proprietary integration. Cross-partner composition through CKS standard interfaces is architecturally different from custom integration code written for a specific pair of partners. The distinction matters for the prior-art posture: proprietary integration does not produce the architectural pattern this note formalizes.

Cross-partner composition does not bypass A1.03 conflict handling. Cross-partner conflicts — including A6.12 multi-author rule authoring conflicts — are first-class architectural artifacts governed by the same conflict-preservation and conflict-resolution architecture that governs single-partner conflicts. Cross-partner status does not create a parallel or bypass conflict channel.

Cross-partner modularity is not inter-Self coordination per Paper 3. Paper 2 governs intra-Self architecture. Cross-partner composition refers to arrangements among composition partners within Paper 1's A1.16 framework — different deployers combining their CKS systems, with each deployer's system operating as a CKS deployment. Inter-Self coordination, in which distinct CKS Selves exchange substrate content across a shared inter-Self substrate, is the subject of Paper 3 and is out of scope for Paper 2 and for this note. The architectural distinction matters: cross-partner composition involves composition of cells and aspects into shared intra-Self structures; inter-Self coordination involves exchange between fully constituted Selves across a shared substrate boundary.

8. One-sentence test

A CKS deployment instantiates modularity across cross-partner compositions if and only if: cells and aspects from different deployment partners compose into shared structures through the standard A1.02 boundary and A1.13 requirements that apply to all CKS compositions; the A2.47 authority distribution specifies who holds cross-partner authority over each shared component before cross-partner operations proceed; the applicable A2.92–A2.94 pattern is identified for each shared structure; cross-partner rule

authoring conflicts are treated per A6.12 as first-class substrate state; each partner retains A1.01 governance over their own components; and cross-partner operations carry A2.40 provenance attribution to the originating partner.

9. Why naming this variant as standalone matters; position in the B1.08 decomposition

The B1.08 decomposition across B2.35 through B2.39 articulates the full operational content of CKS modularity as an architectural commitment. B2.35 establishes that modularity is a consequence of the architecture rather than an engineering achievement. B2.36 establishes that modularity enables composition configurability — the same cells can be configured into different aspect arrangements without interface renegotiation. B2.37 establishes that modularity enables vertical evolution — structural reorganization proceeds without full-system replacement because the modular interface remains stable. B2.38, this note, establishes that modularity extends across composition partners through standard interfaces, with A2.47 authority distribution supplying the governance layer and A6.12 conflict treatment preserving cross-partner conflicts as first-class artifacts. B2.39 will close the B1.08 decomposition by formalizing modularity verification — how a deployment confirms that its modular architecture satisfies the commitments B1.08 names.

Each note in this decomposition claims a distinct territory in the prior-art landscape. B2.38 claims the specific territory of cross-partner modular composition under federated governance: cells from different partners composing through standard interfaces, with A2.47 authority distribution governing shared components and A6.12 treating cross-partner conflicts. Any future system that implements this precise configuration — without changing it materially — will encounter this public prior-art record. That is the strategic purpose the note serves, and the reason naming it as a standalone derivation, rather than leaving it as an implicit consequence of B1.08, is essential to the defensive publication posture.

Series B — Paper 2 Derivation Notes. This note is published as public prior art under CC BY 4.0. Cite as: Li, W. (2026). Modularity Across Cross-Partner Compositions: Formalizing How CKS Modular Architecture Operates Across Deployment Partners per A1.16 and A2.47. Derivation Note B2.38, CKS Theory Series.